

Database Management Systems

Study notes: concepts, relational design, SQL, transactions, performance, and review questions

A practical reference for learning DBMS fundamentals and answering common database questions.

1. DBMS Fundamentals

Database and DBMS

A database is an organized collection of related data. A Database Management System (DBMS) is software that defines, creates, stores, retrieves, updates, and protects that data. Examples include MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, and SQLite.

A database system includes the database, the DBMS software, applications, users, procedures, and the hardware that stores and runs them. A DBMS is more than a file: it provides controlled, reliable access to data.

Why use a DBMS?

A DBMS reduces unnecessary duplication, supports consistent updates, lets multiple users share data, enforces rules, controls permissions, provides backup and recovery, and offers data independence. It also adds administration and operational cost, and can be excessive for a tiny single-user task.

A database schema is the designed structure of a database. A database instance is the actual data stored at a particular moment. DDL changes schema; INSERT, UPDATE, and DELETE change the instance.

2. Architecture and Data Independence

Three-schema architecture

The external level contains user-specific views; the conceptual level describes the complete logical database; and the internal level describes files, pages, indexes, and physical storage. Mappings connect the levels.

Data independence means changing one level without requiring changes at the next higher level. Physical data independence allows storage or index changes without changing the logical schema. Logical data independence allows some conceptual-schema changes without changing user views or application programs; it is generally harder to achieve.

Database languages and users

A DBA manages authorization, availability, backup, performance, and schema changes. Database designers model data and constraints. Developers build applications and queries. End users interact through applications or SQL tools.

A data dictionary or system catalog stores metadata such as tables, columns, data types, constraints, indexes, and privileges.

3. Data Models and ER Design

A data model describes how data, relationships, and constraints are represented. The relational model uses tables; the hierarchical model uses a tree; the network model allows graph-like links; object-oriented models represent objects; and document, key-value, column-family, and graph models are common NoSQL approaches.

Entity-relationship model

An entity is a distinguishable real-world object, such as Student. An entity set is a collection of similar entities. An attribute describes an entity, such as student_id or email. A relationship connects entities, such as Student enrolls in Course.

A key attribute identifies an entity. A composite attribute can be divided into parts; a multivalued attribute can have several values; and a derived attribute can be calculated from other data. In a relational design, multivalued attributes are commonly moved to a separate table.

Relationship cardinality describes how many entities may participate: one-to-one (1:1), one-to-many (1:N), or many-to-many (M:N). Participation may be total (every entity must participate) or partial (participation is optional).

To map an ER model to relations, create a table for each strong entity, put a foreign key on the many side of a 1:N relationship, and create a junction table for an M:N relationship. The junction table normally has foreign keys to both

participating tables and a composite primary key.

4. Relational Model, Keys, and Constraints

A relation is a table, a tuple is a row, and an attribute is a column. A domain is the set of valid values for an attribute. In the formal relational model, rows are unordered and duplicate tuples are not allowed; SQL tables can contain duplicates unless constraints prevent them.

Key types

A superkey is any attribute set that uniquely identifies a row. A candidate key is a minimal superkey. One candidate key is chosen as the primary key; remaining candidate keys are alternate keys. A composite key uses multiple columns. A foreign key references a candidate or primary key in another table (or the same table).

A surrogate key is an artificial identifier, often an integer or UUID. A natural key comes from meaningful business data, such as a carefully chosen registration number. A primary key should be unique, stable, and non-null.

Integrity constraints

Entity integrity requires primary-key values to be unique and not NULL. Referential integrity requires each non-NULL foreign-key value to match a referenced key. Domain constraints restrict allowed values and types. CHECK, NOT NULL, UNIQUE, PRIMARY KEY, and FOREIGN KEY constraints express these rules in SQL.

5. SQL Essentials

SQL is the standard language used with relational databases. Exact syntax and supported features vary by DBMS. Common command categories are: DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE), query/DQL (SELECT), DCL (GRANT, REVOKE), and TCL (COMMIT, ROLLBACK, SAVEPOINT). Some references classify SELECT as DML.

Create tables and add data

```
CREATE TABLE Student (  
    student_id INTEGER PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    department_id INTEGER,  
    gpa DECIMAL(3,2) CHECK (gpa BETWEEN 0 AND 4),  
    FOREIGN KEY (department_id) REFERENCES Department(department_id)  
);
```

```
INSERT INTO Student (student_id, name, email, department_id, gpa)  
VALUES (101, 'Asha Rao', 'asha@example.edu', 10, 3.70);
```

Filter and sort

```
SELECT name, gpa  
FROM Student  
WHERE gpa >= 3.50 AND department_id = 10  
ORDER BY gpa DESC, name ASC;
```

WHERE filters rows before grouping. Use IS NULL or IS NOT NULL to test nulls; comparisons such as = NULL do not work as intended. AND, OR, and NOT combine conditions. LIKE supports patterns (% means any-length text; _ means one character).

Aggregate and group

```
SELECT department_id, COUNT(*) AS student_count, AVG(gpa) AS average_gpa  
FROM Student  
GROUP BY department_id  
HAVING COUNT(*) >= 5;
```

COUNT, SUM, AVG, MIN, and MAX are aggregate functions. GROUP BY forms groups; HAVING filters groups after aggregation, while WHERE filters input rows before grouping. COUNT(*) counts rows; COUNT(column) ignores NULL values in that column.

Joins

```
SELECT s.name, d.department_name
FROM Student AS s
JOIN Department AS d
  ON s.department_id = d.department_id;
```

```
SELECT s.name, d.department_name
FROM Student AS s
LEFT JOIN Department AS d
  ON s.department_id = d.department_id;
```

An INNER JOIN returns matching row pairs. A LEFT JOIN returns all left-side rows and matching right-side data, using NULL where there is no match. RIGHT JOIN preserves the right side; FULL OUTER JOIN preserves both sides where supported. A CROSS JOIN returns combinations of every row pair. A self-join joins a table to itself.

Subqueries, views, and NULL

A subquery is a query nested inside another query. EXISTS tests whether the subquery returns a row. A view is a named query that presents data like a virtual table; a materialized view stores query results and must be refreshed according to the DBMS.

SQL uses three-valued logic: TRUE, FALSE, and UNKNOWN. Comparisons involving NULL usually produce UNKNOWN. Use COALESCE(value, fallback) to substitute for NULL. Be explicit about NULL behavior in filters and aggregates.

6. Functional Dependencies and Normalization

Normalization organizes relations to reduce avoidable duplication and modification anomalies. A functional dependency $X \rightarrow Y$ means that if two rows agree on attributes X, they must agree on attributes Y. A determinant is the left side X.

An insertion anomaly makes it difficult to add one fact without another. An update anomaly requires the same fact to be changed in multiple rows. A deletion anomaly removes a useful fact when deleting an unrelated row.

Normal forms

First Normal Form (1NF): each cell contains one value from its domain, with no repeating groups. The interpretation of atomic depends on the application.

Second Normal Form (2NF): the relation is in 1NF and every non-prime attribute is fully dependent on every candidate key, not just part of a composite candidate key. A non-prime attribute is not part of any candidate key.

Third Normal Form (3NF): for every nontrivial functional dependency $X \rightarrow A$, X is a superkey or A is a prime attribute. Informally, this removes problematic transitive dependencies of non-key attributes on a key.

Boyce-Codd Normal Form (BCNF): for every nontrivial functional dependency $X \rightarrow Y$, X is a superkey. BCNF is stricter than 3NF and can sacrifice dependency preservation in some decompositions.

A lossless decomposition can be joined to recover exactly the original relation without spurious rows. Dependency preservation means constraints can be checked on decomposed tables without joining them. Good normalization aims for lossless decomposition and, when possible, dependency preservation.

Normalization is not simply splitting every table. Denormalization may be chosen for measured performance or reporting needs, but it adds consistency work and should be deliberate.

7. Transactions and Concurrency

A transaction is a logical unit of work. It either commits its intended changes or, when rolled back, has its uncommitted changes undone.

ACID properties

Atomicity: all transaction operations happen or none do. Consistency: a transaction preserves declared rules and valid application invariants. Isolation: concurrent transactions behave according to the configured isolation guarantees.

Durability: committed changes survive failures covered by the system design.

Concurrency control

Concurrency allows multiple transactions to overlap. Without control, problems can include a lost update (one write overwrites another), a dirty read (reading uncommitted data), a non-repeatable read (the same row changes between reads), and a phantom (a repeated predicate query returns a different set of rows).

A schedule is serializable if its result is equivalent to some serial ordering of the transactions. Two-phase locking (2PL) has a growing phase, when locks are acquired, and a shrinking phase, when locks are released; strict 2PL holds write locks until commit or rollback. Locking can cause deadlocks, which systems detect or prevent by aborting or waiting on a transaction.

Isolation levels commonly include Read Uncommitted, Read Committed, Repeatable Read, and Serializable. Exact anomaly guarantees and implementation differ by DBMS; snapshot isolation and MVCC are also common. MVCC keeps versions so readers can often proceed without blocking writers, but it does not eliminate every conflict.

Transaction example

```
BEGIN;  
UPDATE Account SET balance = balance - 50 WHERE account_id = 1;  
UPDATE Account SET balance = balance + 50 WHERE account_id = 2;  
COMMIT;  
  
-- Use ROLLBACK instead of COMMIT if validation fails.
```

8. Indexes and Query Performance

An index is an auxiliary data structure that helps find rows without scanning an entire table. It can speed up selective filters, joins, and ordering, but consumes storage and adds work to inserts, updates, and deletes.

A B+ tree index keeps keys ordered and supports equality and range searches. A hash index is suited mainly to equality lookups and generally not range ordering. Availability and behavior depend on the DBMS.

A clustered index determines or closely corresponds to the physical row order in systems that support that concept; a table generally has only one such ordering. A nonclustered index stores keys and row references separately. These terms vary across DBMS products.

A composite index on (A, B) is typically most useful for conditions beginning with A, such as A or A and B; it may not help a condition only on B. Index column order matters. A covering index contains all values needed by a query, avoiding some table lookups.

Use an execution plan to see scans, index access, join methods, estimated rows, and costs. Measure with representative data. Avoid adding indexes blindly, selecting unnecessary columns, or wrapping indexed columns in expressions that prevent efficient access when the optimizer cannot use the index.

9. Storage, Query Processing, and Recovery

A database commonly moves data between persistent storage and memory in pages or blocks. The buffer manager caches pages in memory. The query processor parses and validates SQL, chooses a plan using optimizer estimates, and executes operators such as scans, filters, joins, sorts, and aggregates.

Failure recovery

A log records changes needed to recover after failures. Write-ahead logging (WAL) requires the relevant log record to reach stable storage before the changed data page is written. Recovery can redo committed changes and undo incomplete transactions, depending on the design.

A checkpoint records a recovery position or state so recovery need not scan the entire log. Backups provide a separate recovery copy; a transaction log alone is not a substitute for tested backups. Recovery planning should define acceptable data loss (RPO) and restoration time (RTO), and backups should be periodically tested.

10. Database Types and Distributed Data

A relational DBMS stores data in related tables and commonly uses SQL and constraints. A document database stores document-shaped records; a key-value database maps keys to values; a column-family database groups data by column families; a graph database models nodes and edges.

NoSQL describes several non-relational database families, not one single model. These systems may suit flexible schemas, large distributed workloads, or graph-shaped relationships. Trade-offs include consistency semantics, query capabilities, transactions, duplication, and operational complexity. Choose by access patterns and requirements, not by trend.

In distributed systems, replication keeps copies of data on multiple nodes and can improve availability or read capacity. Sharding partitions data across nodes for scale. Replication and sharding are different: replicas copy data; shards divide it. Both introduce failure, routing, consistency, and rebalancing considerations.

CAP is often summarized for distributed data: during a network partition, a system must choose between consistency and availability for affected operations. It does not mean a system permanently chooses only two of consistency, availability, and partition tolerance; partition tolerance matters when partitions occur, and real guarantees are more nuanced.

11. Security and Administration

Authentication verifies identity; authorization determines allowed actions. Apply least privilege by granting each user only the permissions needed. Roles group privileges. Protect data in transit with secure connections and at rest with appropriate encryption and key management.

Use parameterized queries or prepared statements for user-supplied values; do not build SQL by concatenating untrusted input. This reduces SQL injection risk. Validate input, protect credentials, audit sensitive actions, patch database software, and restrict network access.

Backups, replication, and high availability solve related but different problems. Replication can copy accidental deletion or corruption; it is not a backup by itself. Keep independent backups and test restoration procedures.

12. Common DBMS Questions and Answers

What is the difference between DBMS and RDBMS?

DBMS is the broad term for software that manages databases. An RDBMS is a DBMS based on the relational model, representing data in tables and supporting relationships and constraints. Most modern relational systems also support features beyond the basic model.

What is the difference between a primary key and a foreign key?

A primary key uniquely identifies a row in its own table and cannot be NULL. A foreign key refers to a key in another table or the same table, enforcing a relationship; it may allow NULL unless constrained otherwise.

What is the difference between DELETE, TRUNCATE, and DROP?

DELETE removes selected rows and can use WHERE. TRUNCATE removes all rows efficiently in many systems, with transaction and identity behavior depending on the DBMS. DROP removes the database object itself. Exact logging, rollback, trigger, and permission behavior is product-specific.

What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping. HAVING filters groups after GROUP BY and can use aggregate conditions.

What is the difference between a candidate key and a superkey?

Both uniquely identify rows. A candidate key is minimal: removing any attribute would make it stop being unique. A superkey may contain unnecessary attributes.

What is a deadlock?

A deadlock occurs when transactions wait in a cycle for resources held by one another. A DBMS may detect the cycle and abort a victim, or use prevention and timeout strategies.

What is a view?

A view is a named query presented as a virtual table. It can simplify queries and restrict exposed columns or rows. A regular view usually stores its definition, not the result; a materialized view stores results.

When should an index be created?

Consider an index when measured workloads frequently filter, join, or sort on its columns and the expected read benefit justifies storage and write overhead. Confirm with query plans and realistic measurements.

What is the difference between normalization and denormalization?

Normalization structures data to reduce duplication and anomalies. Denormalization deliberately duplicates or precomputes data to improve specific reads, at the cost of additional storage and consistency work.